Part 4: Squashing the Decryption Circuit and Bootstrapping

Fully Homomorphic Encryption over the Integers

Nirav Bhattad

March 27, 2026

Outline: Part 4

1. The Bootstrapping Blueprint
2. The Depth Problem
3. Squashing the Decryption Circuit
4. The Shallow Decryption Circuit
5. References

The Crisis of the Noise Bottleneck

The Problem: Our Somewhat Homomorphic Encryption (SHE) scheme can add and multiply. However, every multiplication squares the noise.

The Crisis of the Noise Bottleneck

The Problem: Our Somewhat Homomorphic Encryption (SHE) scheme can add and multiply. However, every multiplication squares the noise.

Eventually, the noise exceeds the decryption boundary of $p/2$, and the ciphertext is permanently destroyed. How can we possibly evaluate a circuit of arbitrary depth?

The Crisis of the Noise Bottleneck

The Problem: Our Somewhat Homomorphic Encryption (SHE) scheme can add and multiply. However, every multiplication squares the noise.

Eventually, the noise exceeds the decryption boundary of $p/2$, and the ciphertext is permanently destroyed. How can we possibly evaluate a circuit of arbitrary depth?

The Solution: Bootstrapping (Gentry, 2009) If an encryption scheme is capable of homomorphically evaluating *its own decryption circuit*, plus one additional gate (like a NAND gate), it can evaluate *any* circuit of *any* depth.

How Bootstrapping Works

The Bootstrapping Mechanism:

1. Take a "noisy" ciphertext c (where noise is perilously close to $p/2$).
2. Encrypt c *again* under a new public key, resulting in a doubly-encrypted ciphertext.
3. Provide the server with an *encrypted* version of the secret key sk.
4. The server homomorphically evaluates the decryption algorithm on the inner ciphertext using the encrypted secret key.

How Bootstrapping Works

The Bootstrapping Mechanism:

1. Take a "noisy" ciphertext c (where noise is perilously close to $p/2$).
2. Encrypt c *again* under a new public key, resulting in a doubly-encrypted ciphertext.
3. Provide the server with an *encrypted* version of the secret key sk.
4. The server homomorphically evaluates the decryption algorithm on the inner ciphertext using the encrypted secret key.

Intuition: Because we evaluate the decryption circuit homomorphically, the output is not the raw plaintext. Instead, it is a **fresh ciphertext** encrypting the exact same message!
This new ciphertext has a fixed, smaller amount of noise determined solely by the depth of the decryption circuit. We have successfully "refreshed" the ciphertext.

Defining Bootstrappability

Definition (1.1: Bootstrappable Encryption)

Let $\mathcal{E}$ be a homomorphic encryption scheme, and let $\mathcal{C}_{\mathcal{E}}$ be the set of permitted circuits it can evaluate before noise exceeds the bounds.
Let $\mathcal{D}_{\mathcal{E}}$ be the set of augmented decryption circuits (decryption plus one additional gate).
We say $\mathcal{E}$ is **bootstrappable** if:

$$\mathcal{D}_{\mathcal{E}} \subseteq \mathcal{C}_{\mathcal{E}}$$

Defining Bootstrappability

Definition (1.1: Bootstrappable Encryption)

Let $\mathcal{E}$ be a homomorphic encryption scheme, and let $\mathcal{C}_{\mathcal{E}}$ be the set of permitted circuits it can evaluate before noise exceeds the bounds.
Let $\mathcal{D}_{\mathcal{E}}$ be the set of augmented decryption circuits (decryption plus one additional gate).
We say $\mathcal{E}$ is **bootstrappable** if:

$$\mathcal{D}_{\mathcal{E}} \subseteq \mathcal{C}_{\mathcal{E}}$$

In plain English: A scheme is bootstrappable if its noise budget is large enough to survive the homomorphic evaluation of its own decryption algorithm.

The Depth Problem: Integer Division

Can our SHE scheme handle its own decryption circuit? Let's look at the math.

The Standard Decryption Equation:

$$m' = (c \pmod{p}) \pmod{2}$$

The Depth Problem: Integer Division

Can our SHE scheme handle its own decryption circuit? Let's look at the math.

The Standard Decryption Equation:

$$m' = (c \pmod{p}) \pmod{2}$$

To evaluate this homomorphically as a boolean circuit, we must compute it over the integers:

$$c \pmod{p} = c - p\lfloor c/p \rceil$$

The Depth Problem: Integer Division

Can our SHE scheme handle its own decryption circuit? Let's look at the math.

The Standard Decryption Equation:

$$m' = (c \pmod{p}) \pmod{2}$$

To evaluate this homomorphically as a boolean circuit, we must compute it over the integers:

$$c \pmod{p} = c - p\lfloor c/p \rceil$$

The Roadblock: This requires computing $\lfloor c/p \rceil$, which relies on **integer division**. When translated into a boolean polynomial circuit, division has a massive multiplicative depth.

The Depth Problem: Integer Division

Can our SHE scheme handle its own decryption circuit? Let's look at the math.

The Standard Decryption Equation:

$$m' = (c \pmod{p}) \pmod{2}$$

To evaluate this homomorphically as a boolean circuit, we must compute it over the integers:

$$c \pmod{p} = c - p\lfloor c/p \rceil$$

The Roadblock: This requires computing $\lfloor c/p \rceil$, which relies on **integer division**. When translated into a boolean polynomial circuit, division has a massive multiplicative depth.

Our permitted circuits in the SHE scheme can only handle polynomials up to degree:

$$d \leq \frac{\eta - 4 - \log ||\hat{f}||_1}{\rho' + 2}$$

The standard division circuit's degree far exceeds this bound. We must **"squash"** the decryption circuit.

Squashing: Shifting the Burden

To make the scheme bootstrappable, we must reduce the multiplicative depth of the decryption circuit.

Squashing: Shifting the Burden

To make the scheme bootstrappable, we must reduce the multiplicative depth of the decryption circuit.

The Strategy: We shift the heavy computational lifting from the Decrypt algorithm (which is strictly constrained by the noise budget) to the Evaluate algorithm (which can be done in the clear by the server *before* the final homomorphic decryption step).

Squashing: Shifting the Burden

To make the scheme bootstrappable, we must reduce the multiplicative depth of the decryption circuit.

The Strategy: We shift the heavy computational lifting from the Decrypt algorithm (which is strictly constrained by the noise budget) to the Evaluate algorithm (which can be done in the clear by the server *before* the final homomorphic decryption step).

New Parameters:

- κ: Precision of rational numbers.
- Θ: Size of a public set of rational numbers.
- θ: The Hamming weight of a sparse secret subset.

Modifying the Key Generation

During KeyGen, alongside p and pk^*, we generate a "hint" about $1/p$.

Modifying the Key Generation

During KeyGen, alongside p and pk^*, we generate a "hint" about $1/p$.

1. The Sparse Secret Key: Choose a random Θ-bit vector $\vec{s} = \langle s_1, \ldots, s_\Theta \rangle$ with a very low Hamming weight θ. Let $S = \{i : s_i = 1\}$. This vector $\vec{s}$ becomes the new secret key.

Modifying the Key Generation

During KeyGen, alongside p and pk^*, we generate a "hint" about $1/p$.

1. The Sparse Secret Key: Choose a random Θ-bit vector $\vec{s} = \langle s_1, \ldots, s_\Theta \rangle$ with a very low Hamming weight θ. Let $S = \{i : s_i = 1\}$. This vector $\vec{s}$ becomes the new secret key.

2. The Public Rational Numbers: Generate a set $\vec{y} = \{y_1, \ldots, y_\Theta\}$ of rational numbers in $[0, 2)$ with κ bits of precision, such that the subset S sums to approximately $1/p$:

$$\sum_{i \in S} y_i \approx \frac{1}{p} \pmod 2$$

Specifically, $\left[\sum_{i \in S} y_i\right]_2 = \frac{1}{p} - \Delta_p$, where $|\Delta_p| < 2^{-\kappa}$.

Modifying the Key Generation

During KeyGen, alongside p and pk^*, we generate a "hint" about $1/p$.

1. The Sparse Secret Key: Choose a random Θ-bit vector $\vec{s} = \langle s_1, \ldots, s_\Theta \rangle$ with a very low Hamming weight θ. Let $S = \{i : s_i = 1\}$. This vector $\vec{s}$ becomes the new secret key.

2. The Public Rational Numbers: Generate a set $\vec{y} = \{y_1, \ldots, y_\Theta\}$ of rational numbers in $[0, 2)$ with κ bits of precision, such that the subset S sums to approximately $1/p$:

$$\sum_{i \in S} y_i \approx \frac{1}{p} \pmod 2$$

Specifically, $\left[\sum_{i \in S} y_i\right]_2 = \frac{1}{p} - \Delta_p$, where $|\Delta_p| < 2^{-\kappa}$.

Intuition: We hide a sparse subset sum within the public key. By giving the server these y_i fragments, the server can multiply the ciphertext by $1/p$ for us, saving the decryption circuit from doing the division!

Modifying Evaluate and Decrypt

When the server finishes evaluating a circuit, it holds a noisy ciphertext c^*. Before returning it, the server does post-processing in the clear.

Modifying Evaluate and Decrypt

When the server finishes evaluating a circuit, it holds a noisy ciphertext c^*. Before returning it, the server does post-processing in the clear.

Server Post-Processing: For $i \in \{1, \ldots, \Theta\}$, compute:

$$z_i = [c^* \cdot y_i]_2$$

keeping only $n = \lceil \log \theta \rceil + 3$ bits of precision. The server outputs c^* and $\vec{z} = \langle z_1, \ldots, z_\Theta \rangle$.

Modifying Evaluate and Decrypt

When the server finishes evaluating a circuit, it holds a noisy ciphertext c^*. Before returning it, the server does post-processing in the clear.

Server Post-Processing: For $i \in \{1, \ldots, \Theta\}$, compute:

$$z_i = [c^* \cdot y_i]_2$$

keeping only $n = \lceil \log \theta \rceil + 3$ bits of precision. The server outputs c^* and $\vec{z} = \langle z_1, \ldots, z_\Theta \rangle$.

Deriving the Squashed Decryption: We need to compute $c^*/p \pmod 2$.

$$\frac{c^*}{p} \approx c^* \sum_{i \in S} y_i \pmod 2$$

$$= \sum_{i=1}^{\Theta} s_i (c^* y_i) \pmod 2 \approx \sum_{i=1}^{\Theta} s_i z_i \pmod 2$$

Modifying Evaluate and Decrypt

When the server finishes evaluating a circuit, it holds a noisy ciphertext c^*. Before returning it, the server does post-processing in the clear.

Server Post-Processing: For $i \in \{1, \ldots, \Theta\}$, compute:

$$z_i = [c^* \cdot y_i]_2$$

keeping only $n = \lceil \log \theta \rceil + 3$ bits of precision. The server outputs c^* and $\vec{z} = \langle z_1, \ldots, z_\Theta \rangle$.

Deriving the Squashed Decryption: We need to compute $c^*/p \pmod 2$.

$$\frac{c^*}{p} \approx c^* \sum_{i \in S} y_i \pmod 2$$

$$= \sum_{i=1}^{\Theta} s_i (c^* y_i) \pmod 2 \approx \sum_{i=1}^{\Theta} s_i z_i \pmod 2$$

The New Decrypt Algorithm:

$$m' = \left[c^* - \left\lfloor \sum_{i=1}^{\Theta} s_i z_i \right\rceil \right]_2$$

The Climax: The Shallow Circuit

Look at our new decryption equation: $m' = \left[c^* - \lfloor \sum_{i=1}^{\Theta} s_i z_i \rceil \right]_2$.

Why is this brilliant?

- The server already gave us the rational numbers z_i.
- The secret key bits s_i are just 0 or 1.
- Our only job is to sum up numbers, and because $\vec{s}$ is sparse, only θ of them are non-zero!

The Climax: The Shallow Circuit

Look at our new decryption equation: $m' = \left[c^* - \lfloor \sum_{i=1}^{\Theta} s_i z_i \rceil\right]_2$.

Why is this brilliant?

- The server already gave us the rational numbers z_i.
- The secret key bits s_i are just 0 or 1.
- Our only job is to sum up numbers, and because $\vec{s}$ is sparse, only θ of them are non-zero!

Lemma (4.1: Elementary Symmetric Polynomials)

We use the "three-for-two" trick to reduce the sum. The i-th bit of the Hamming weight of a binary vector $\vec{\sigma}$ can be expressed as a polynomial of degree exactly 2^i using the 2^i-th elementary symmetric polynomial $e_{2^i}(\vec{\sigma})$.

The Climax: The Shallow Circuit

Look at our new decryption equation: $m' = \left[c^* - \lfloor \sum_{i=1}^{\Theta} s_i z_i \rceil\right]_2$.

Why is this brilliant?

- The server already gave us the rational numbers z_i.
- The secret key bits s_i are just 0 or 1.
- Our only job is to sum up numbers, and because $\vec{s}$ is sparse, only θ of them are non-zero!

Lemma (4.1: Elementary Symmetric Polynomials)

We use the "three-for-two" trick to reduce the sum. The i-th bit of the Hamming weight of a binary vector $\vec{\sigma}$ can be expressed as a polynomial of degree exactly 2^i using the 2^i-th elementary symmetric polynomial $e_{2^i}(\vec{\sigma})$.

Because we only need $\lceil \log \theta \rceil + 3$ bits of precision, the maximum degree required to compute the binary sum and rounding is bounded:

$$\text{Degree} \leq 64\theta \log^2 \theta$$

Bootstrapping Achieved

By setting $\theta = \lambda$, the degree of our squashed decryption polynomial is $\mathcal{O}(\lambda \log^2 \lambda)$.

Bootstrapping Achieved

By setting $\theta = \lambda$, the degree of our squashed decryption polynomial is $\mathcal{O}(\lambda \log^2 \lambda)$.

Theorem (4.2: Bootstrapping Achieved)

By setting the secret key length $\eta = \rho \cdot \Theta(\lambda \log^2 \lambda)$, the base SHE scheme can comfortably handle polynomials of degree $\mathcal{O}(\lambda \log^2 \lambda)$.
Because the degree of the squashed decryption circuit is strictly less than the maximum degree of the permitted polynomials, we have:

$$\mathcal{D}_{\mathcal{E}} \subset \mathcal{C}_{\mathcal{E}}$$

Therefore, the scheme is bootstrappable and, consequently, **Fully Homomorphic**.

Bootstrapping Achieved

By setting $\theta = \lambda$, the degree of our squashed decryption polynomial is $\mathcal{O}(\lambda \log^2 \lambda)$.

Theorem (4.2: Bootstrapping Achieved)

By setting the secret key length $\eta = \rho \cdot \Theta(\lambda \log^2 \lambda)$, the base SHE scheme can comfortably handle polynomials of degree $\mathcal{O}(\lambda \log^2 \lambda)$.
Because the degree of the squashed decryption circuit is strictly less than the maximum degree of the permitted polynomials, we have:

$$\mathcal{D}_{\mathcal{E}} \subset \mathcal{C}_{\mathcal{E}}$$

Therefore, the scheme is bootstrappable and, consequently, **Fully Homomorphic**.

Conclusion: By publishing an approximation of $1/p$ and forcing the evaluator to pre-compute partial products, we squashed decryption down to a simple sparse subset sum. We have achieved FHE using nothing but modular arithmetic over the integers.

References

- Marten van Dijk, Craig Gentry, Shai Halevi, and Vinod Vaikuntanathan. *Fully Homomorphic Encryption over the Integers*. Eurocrypt, 2010.
- Craig Gentry. *A Fully Homomorphic Encryption Scheme*. PhD Thesis, Stanford University, 2009.